

RELATÓRIO TÉCNICO DE SEGURANÇA

Relatório de Auditoria de Segurança

atelié-erp

Ateliê ERP + DeskcommCRM — Astro 7 SSR · Cloudflare Workers · Cloudflare D1 · TypeScript

DATA 02 de outubro de 2026	ACHADOS 19	CATEGORIAS 5	ROTAS AUDITADAS 52 arquivos em src/pages/api
-------------------------------	---------------	-----------------	---

Escopo auditado

- Backend.** Os 52 arquivos de `src/pages/api/**` (todas as rotas, sem amostragem), as fábricas `routeFactory.ts` e `crud.ts`, e a camada de autenticação/autorização `middleware.ts`, `auth.ts`, `authz.ts`, `origin.ts`, `rateLimit.ts`.
- Frontend.** Todos os `src/ui/views/**` e `src/ui/**`, os layouts `.astro` e a política de escape (`escapeHtml` / `escapeAtrib` / `escapeAttr`) e a CSP.
- Segredos.** `wrangler.toml`, `.dev.vars`, `.dev.vars.example`, `waha/docker-compose.waha.yml`, `.github/workflows/ci.yml`, `migrations/*.sql`, `scripts/`, documentação e o **histórico git completo** (55 commits).

Nota metodológica — mapeamento das categorias para esta stack

A stack detectada é **Astro 7** em `output: 'server'` sobre **Cloudflare Workers**, com persistência em **Cloudflare D1** (SQLite) acessado por **SQL parametrizado escrito à mão** (`sql.ts/crud.ts`) — **sem ORM e sem query builder**. O frontend é **framework-free**: DOM imperativo + CSS puro, sem React, o que elimina a categoria de sinks `dangerouslySetInnerHTML` e transfere o XSS do frontend para `innerHTML` + Astro `set:html`. A autenticação é **sessão em D1 com token Bearer**, não cookie nem OAuth, e não existe Supabase no app auditado. Cada categoria foi endereçada assim:

Categoria pedida	Equivalente nesta stack
1. Banco sem trança	Não há RLS (o projeto não usa Supabase). O mecanismo de isolamento real foi identificado como sendo <i>a ausência de isolamento</i> : <code>schema single-tenant</code> , <code>listEntities = SELECT *</code> , autorização por papel em <code>authz.ts</code> . Auditamos listagem, busca, agregação, relatório e exportação.
2. Permissão no navegador	Gate de papel existe em <code>authz.ts:requireRole</code> e é cruzado, um a um, com os gates do frontend (<code>isAdmin/canEdit/role</code>) e com o handler de cada endpoint.
3. IDOR	Todos os handlers com <code>:id</code> em path, query ou body foram percorridos (não amostras), rastreando até a tabela do D1 se há filtro de posse.
4. Chaves expostas	Código, config, <code>[vars]</code> do <code>wrangler</code> , <code>.dev.vars</code> , <code>docker-compose</code> , CI, <code>migrations (seed)</code> , <code>scripts</code> , docs e histórico git (<code>git log -S</code>). Defaults inseguros <code>\${VAR:-default}</code> e ausência de validação de startup foram checados.
5. Inputs sem tratamento	Inventário de todos os sinks <code>innerHTML/outerHTML/insertAdjacentHTML/set:html</code> , com rastreamento da <i>origem</i> de cada valor interpolado (API, storage, query string, digitação) e verificação de biblioteca de sanitização.

Relatório gerado por `docs/security-audit/gerar-relatorio.py` — regerável a qualquer momento. Todos os achados foram verificados no código real, com arquivo e linha.

Resumo executivo

Foram registrados **19 achados** em cinco categorias. O perfil é de uma base com **fundamentos sólidos e uma lacuna estrutural**: a autenticação, a criptografia de senha, o rate limit, a CSP, a auditoria e o transporte do WAHA estão bem implementados e verificados. O problema está na **camada de autorização** — e ele se origina em um único lugar: as fábricas que geram as rotas de entidade não recebem papéis, então o único controle real é "ter sessão". Disso decorrem a escalada de privilégio (crítica), a escrita por papel de leitura, e o IDOR em ~24 rotas.



Pontos fortes

O que foi verificado e está **correto**. Esta seção é também a prova de cobertura: cada item é um controle que a auditoria procurou e encontrou implementado com evidência.

CONTROLE	EVIDÊNCIA
Recorte LGPD em cascata, no módulo, não na rota src/domain/lgpdScope.ts:69-92	Contatos por atribuição derivam conversas, mensagens e clientes/pedidos. A rota consome o recorte, então o vazamento anterior não voltou por duplo caminho.
SSE escopado na consulta, com escape para manager/admin src/pages/api/crm/events.ts:27-38, 201	<code>`visibleConversationIds`</code> devolve <code>`null`</code> só para manager/admin; nos demais papéis o filtro vai para o SQL. Sem token em query string (único parâmetro lido é <code>`since`</code>).
Sessão em cookie HttpOnly; nada legível por script src/server/auth.ts:70	<code>`HttpOnly; Secure; SameSite=Strict`</code> , com o cliente guardando apenas perfil não-sensível.
Escalada de privilégio fechada nos dois verbos de /api/users src/pages/api/users/index.ts:31-33, 51-58 · [id].ts:62-65	Os papéis aceitos são derivados do corpo antes de qualquer validação; pedir <code>`admin`</code> exige <code>`ADMINS`</code> tanto no POST quanto no PUT.
Matriz de papéis por operação na fábrica de rotas src/server/routeFactory.ts:19-34, 56-60, 204-228	leitura <code>`viewer+`</code> , escrita <code>`agent+`</code> , exclusão <code>`manager+`</code> ; <code>`admin`</code> passa em todas. Escrever deixou de ser synonym de <code>`viewer`</code> .
ROLE_RANK finalmente decide src/server/authz.ts:25-30, 43-50	<code>`hasRank`/`requireRank`</code> comparam <code>`ROLE_RANK`</code> , dando semântica real à hierarquia que antes era código morto.
UI espelha o endpoint em vez de inventar permissão src/ui/views/crm/CrmEquipeView.ts:42-52	O botão de criação é condicionado ao mesmo conjunto de papéis que o POST exige.
settings: leitura pura na escrita, com allowlist src/pages/api/settings.ts:46-62	GET não escreve no banco; PUT exige <code>`admin/manager`</code> e o patch passa por <code>`sanitizeSettingsPatch`</code> antes do merge.
WhatsApp: papel E posse da conversa na escrita src/pages/api/whatsapp/send.ts:28-33, 50-51	Fecha as duas metades — posse sem papel (viewer donando a própria conversa) e papel sem posse (agent em qualquer conversa).
Webhook WAHA: assinatura presente e errada é sempre recusada, comparação em tempo constante src/server/wahaWebhook.ts:69-81, 179-184	<code>`constantTimeEqual`</code> evita vazamento por tempo; corpo bruto arquivado antes de qualquer parse (fail-open deste ponto é C4-05, não daqui).
Anti-SQLi por allowlist de coluna + parâmetros vinculados src/server/mapping.ts:52-58 · src/server/crud.ts:46, 65, 78	<code>`keepAllowed`</code> impede contrabandear nome de coluna via JSON; todo valor vai por <code>`bind()`</code> .
Senhas: PBKDF2-SHA256 com sal por usuário, e sem seed reproduzível src/server/auth.ts · scripts/bootstrap-admin.ts:26-32 · src/server/seedCredential.ts:36-43	Sal sorteado por gravação; o bootstrap sorteia a senha com <code>`crypto.getRandomValues`</code> ; existe função de fail-closed para hash legado (C4-07 registra que falta ligar no boot).
Backbone de autenticação no middleware src/middleware.ts:15-21, 116-136	Todas as rotas <code>`/api/*`</code> exigem sessão, exceto cinco caminhos explícitos em <code>`PUBLIC_PATHS`</code> ; o resto responde 401.
As cinco rotas públicas se auto-autenticam whatsapp/health.ts · whatsapp/webhook-config.ts · crm/events.ts · whatsapp/send.ts	Como o middleware não resolve sessão nelas, cada uma valida o token sozinha — e todas validam (inclusive <code>`events.ts`</code> e <code>`send.ts`</code> , que re-resolvem o usuário).

Rate limit de login em D1, não em memória do isolate src/server/rateLimit.ts · src/middleware.ts:58-64	5 tentativas de login e 3 de troca de senha por janela de 15 min, com contador persistido: resiste a restart de deploy.
Guarda de mesma-origem nas rotas de sessão src/server/origin.ts:10-19	login/logout/me/change-password rejeitam `Origin` divergente; cliente sem header (o engine WAHA) passa.
CSP restritiva e anti-clickjacking src/middleware.ts:31-56	`script-src 'self'` sem `unsafe-inline`, `frame-ancestors 'none'`, `base-uri 'self'`, HSTS, `nosniff`, `X-Frame-Options` — o que contém injeção de HTML.
Auditoria das operações privilegiadas src/server/authz.ts · src/server/routeFactory.ts:42-54 · src/server/audit.ts	Negação por papel gera entrada de auditoria com IP; criação de entidade gera action_log com userId e clientId.
O transporte WAHA nunca carrega segredo no erro src/server/waha.ts:20-21, 40-47	`WahaError` só formata `waha_<op>_<status>`; o corpo da resposta — que pode conter telefone, HMAC e API key — é descartado. Timeout em toda chamada.
Compose do WAHA usa sentinela fail-closed waha/docker-compose.waha.yml:41	`WAHA_API_KEY: "\${WAHA_API_KEY_SHA512:-sha512:INVALID_CHANGE_ME}"` — nenhum hash real casa, então um container recém-criado recusa tudo em vez de abrir com senha padrão.
Trava de regressão de segredo em texto versionado tests/spec-v2/c-settings-secrets.test.ts:232-259	AC-334/AC-335 varrem todo arquivo rastreado e falham se a chave real ou uma atribuição com cara de chave aparecer. Foi exatamente esse teste que reprovou o relatório antigo (C4-01).

Pontos fracos — os riscos centrais

A partir daqui o relatório sai do campo "o que está bem" e entra no campo "o que precisa mudar". Estes quatro pontos resumem a superfície de risco real do aplicativo — os demais achados são consequência deles.

Autorização é a camada fraca

Quatro arquivos no app inteiro chamam `requireRole`. Todas as demais rotas — que são a maioria — confiam apenas no middleware. A hierarquia de papéis foi declarada (`ROLE_RANK`) e nunca usada, e a matriz de permissões está duplicada em literais na UI e no servidor. É exatamente esse arranjo que deixou a escalada de privilégio passar.

"Single-tenant" é uma premissa silenciosa

O app é single-tenant por desenho, e isso está documentado. O risco é que a premissa se propaga sozinha: toda rota nova herdada das fábricas nasce global, e `assignedUserId/assigneeUserId` parecem fronteira de segurança quando são só metadado de atribuição. A rota de privacidade LGPD é a prova de que a distinção importa — e ela falhou (C1-01).

Escape de HTML é aplicado por convenção, não por garantia

Não há biblioteca de sanitização no projeto. O que existe são três helpers de escape, dois deles divergentes, e a aplicação é manual: a maioria das views escapa corretamente, mas `openModal`, o Dashboard, a `LoginView` e os campos numéricos foram esquecidos. Uma regra de lint fecharia essa classe inteira.

Sessão em `localStorage` é o multiplicador

Enquanto o token estiver em `localStorage`, qualquer injeção de HTML que evolua para execução de script vira comprometimento de conta. A CSP atual (`script-src 'self'`) é o que separa "deformar HTML" de "roubar sessão" — um controle único segurando o impacto de quatro achados.

Achados detalhados por categoria

Cada linha cita o arquivo e a linha exatos, o trecho verificado e a condição de explorabilidade. Os achados marcados como *verificação positiva* existem para registrar a cobertura da auditoria.

1. Banco sem trança (isolamento de inquilino/dono)

ID	SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
C1-01	Informativa	src/domain/lgpdScope.ts:69-92 src/pages/api/crm/events.ts:27-38 src/pages/api/crm/events.ts:186 src/server/crud.ts:16 README.md:Modelo de acesso	Verificação positiva: o recorte LGPD filtra mensagens, clientes e pedidos, e o SSE é escapado por conversa O vazamento de PII da rodada anterior está fechado. `scopeForUser` deriva o recorte em cascata: contatos por atribuição → conversas por contato → mensagens por conversa, e clientes por identidade de contato → pedidos por cliente. `GET /api/me/data`, `/api/me/export` e `/api/me/erase` consomem esse recorte, então um `viewer` sem atribuição não recebe mais linhas de terceiros. O SSE segue a mesma disciplina: `visibleConversationIds` devolve `null` (tudo) só para manager/admin e, nos demais papéis, o conjunto de conversas atribuídas — com o filtro aplicado na consulta, não depois. O stream também deixou de transportar o token por query string: o único parâmetro lido na URL é `since`. A listagem global (`SELECT *` sem filtro) permanece, e isso é DECISÃO DE PROJETO documentada no README, não descuido: o app é single-tenant e o controle é por papel. Fica registrado como invariante, não como garantia de isolamento. Explorabilidade: Qualquer sessão válida lê a base inteira nas listagens — aceito por desenho documentado. Já o recorte LGPD e o stream SSE só vazam se alguém remover o filtro do módulo ou da consulta. Correção: Manter. Registrar como teste de invariante: (a) `scopeForUser` nunca devolve linha sem atribuição — testável com um usuário sem atribuição; (b) uma rota nova não pode assumir que `listEntities` é restrita; (c) se um dia existir multi-tenant, `listEntities` é o ponto de entrada obrigatório do filtro. ► trecho de código
C1-02	Informativa	src/server/auth.ts:70 src/repositories/AuthRepository.ts:6-10	Verificação positiva: sessão em cookie HttpOnly; nenhum token legível por script O modelo de ameaça da rodada anterior era: script injetado no origin lê o token e vira sessão válida. Isso não é mais possível — o token vive em cookie `HttpOnly; Secure; SameSite=Strict`, e o cliente guarda apenas o perfil não-sensível do usuário. Como consequência, qualquer regressão de escaping na categoria 5 deixa de ser um elevador direto para sequestro de sessão: o pior caso passa a ser execução no contexto da origem. Explorabilidade: Não explorável isoladamente; é a garantia que limita o impacto de C5-01. Correção: Nenhuma. Preservar e cobrir com teste que falhe se o cookie perder HttpOnly. ► trecho de código

2. Permissão definida no navegador

ID	SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
C2-01	Informativa	src/pages/api/users/index.ts:31-33 src/pages/api/users/index.ts:51-58 src/pages/api/users/[id].ts:62-65	Verificação positiva: POST /api/users deriva os papéis permitidos do corpo (manager não fabrica admin) A escalada de privilégio da rodada anterior está fechada. O gate vem ANTES de qualquer validação de conteúdo, e o conjunto de papéis aceitos é derivado do próprio corpo: pedir `role: 'admin'` exige `ADMINS`. Os dois verbos ficam paritários — POST e PUT chamam funções de AllowedRoles equivalentes, então não existe mais a assimetria em que a rota brother validava papel corretamente e o outro não. Explorabilidade: Sessão `manager` + POST com `role: 'admin'` → 403. Ilegível a `viewer` / `agent`. Correção: Nenhuma. Teste existente já cobre manager→admin nos dois verbos. ► trecho de código
C2-02	Informativa	src/server/routeFactory.ts:19-34 src/server/routeFactory.ts:56-60 src/server/authz.ts:25-30 src/server/authz.ts:43-50	Verificação positiva: matriz de papéis por operação aplicada na fábrica de rotas (ROLE_RANK deixou de ser código morto) O gate de papel deixou de ser uma lista escrita à mão em cada rota. A fábrica declara pisos por operação — leitura `viewer+`, escrita `agent+`, exclusão `manager+` — e um `admin` passa em todas sem repetição. `requireRank` faz a comparação por `ROLE_RANK`, então a hierarquia de papéis, antes declarada e sem consumidor, agora decide de fato. O efeito colateral relevante: escrever não é mais synonym de `viewer`, e apagar não é mais synonym de `agent`. Explorabilidade: Toda rota gerada por createCollectionRoutes/createItemRoutes herda os pisos; rota que declarar RoleConfig próprio sobrescreve. Correção: Nenhuma. Teste que falhe se uma rota nova for criada sem RoleConfig explícito quando o piso padrão mudar. ► trecho de código
C2-03	Informativa	src/ui/views/crm/CrmEquipeView.ts:42-52	Verificação positiva: a UI da Equipe espelha os gates do servidor A divergência UI↔servidor — botão renderizado para papel que o endpoint recusa, e opção `admin` oferecida a quem não pode conceder — está fechada: a decisão de renderizar acontece com o mesmo conjunto de papéis que o endpoint exige. A UI deixou de ser um oráculo de permissão; ela reflete o servidor, que continua sendo a fronteira. Explorabilidade: Qualquer sessão; o efeito é UX (o botão não aparece), não autorização. Correção: Nenhuma. Preservar o comentário que amarra a UI ao endpoint, para a próxima tela não quebrar o espelho. ► trecho de código
C2-04	Informativa	src/pages/api/whatsapp/send.ts:28-33 src/pages/api/whatsapp/send.ts:50-51	Verificação positiva: escrita no WhatsApp exige papel agent+ e posse da conversa Duas deficiências sobrepostas foram fechadas: posse sem papel (viewer donando a própria conversa disparava mensagem para o cliente) e papel sem posse (agent disparava para qualquer conversa). A rota combina as duas, e usa o usuário já resolvido pelo token — coerente com ela ser `PUBLIC_PATHS` e não ter `locals.user`. Explorabilidade: Ilegível sem sessão. Escrita exige papel agent+ e conversa atribuída (ou manager/admin). Correção: Nenhuma. ► trecho de código

C2-05	Informativa	src/pages/api/settings.ts:46-51 src/pages/api/settings.ts:53-62 src/server/mapping.ts:52-58	Verificação positiva: /api/settings é leitura pura na escrita, com allowlist de colunas e papel A rota que permitia escrita em GET, e gravação de coluna arbitrária no PUT, está fechada: o GET não toca mais no banco, o PUT exige `admin/manager` e o patch passa por `sanitizeSettingsPatch` antes do merge. A allowlist de colunas (`keepAllowed`, via `shape.columns`) permanece como segunda barreira contra chave forasteira no corpo — vale notar que ela é o que mantém o contrabando de coluna por JSON fechado no resto do app também. Explorabilidade: PUT exige sessão admin/manager; chave fora do shape é descartada no merge. Correção: Nenhuma. ► trecho de código
-------	-------------	---	---

3. IDOR

ID	SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
C3-01	Alta	src/server/routeFactory.ts:62-69 src/server/routeFactory.ts:81-82 src/server/routeFactory.ts:168-186	IDOR em 15 das 19 rotas de item: agent+ altera registro de terceiro que não tem coluna de dono `OWNER_FIELDS` cobre quatro tabelas (contacts, conversations, deals, tasks) dezenove expostas por `createItemRoutes`. Para as outras quinze, `checkOwnership` retorna `null` na primeira linha — o gate de posse é literalmente ausente, não apenas frouxo. Somado ao piso `update: agent` introduzido em C2-02, o efeito é: qualquer `agent` autenticado faz PUT em qualquer registro de qualquer tabela sem dono, por knowing do id. O caso mais direto é `messages`. A rota é `createItemRoutes(MESSAGES_TABLE, MESSAGES_SHAPE)` — sem RoleConfig, sem dono — e `MESSAGES_SHAPE` inclui `text`, `fromMe`, `createdBy`, `direction`, `conversationId`, `externalId`, `remoteId`, `waStatus`. Ou seja: um `agent` reescreve o texto de uma mensagem já entregue, marca `fromMe: true`, e forja `createdBy`. Isso não é só acesso indevido a registro alheio; é reescrita de histórico de conversação e de autoria. Para contexto, as quatro tabelas com dono estão bem guardadas: manager/admin passam (`:92`), e divergência de dono retorna 403 (`:95-97`). O problema é a cobertura, não o mecanismo — que está correto onde se aplica. Explorabilidade: Sessão `agent` (o papel mais comum do time de vendas) + PUT em `/api/**/[id]` de tabela sem coluna de dono: 200 e a alteração persiste. `viewer` é barrado por C2-02 — a explorabilidade depende de `agent`, não de escalate. Correção: Duas frentes. (1) Subir o piso de `update` para `manager` em tabelas sensíveis sem dono — `messages`, `orders`, `calendar-events`, `conversation-notes`, `catalog-products` — enquanto o modelo de posse não existe. (2) Modelar dono onde faz sentido: acrescentar `assignedUserId` às tabelas de trabalho do CRM (activities, notes, products) com migration e registrar em `OWNER_FIELDS`, e tratar `messages` por posse da conversa (`conversationId` → `conversations.assignedUserId`) em vez de coluna própria. Em qualquer das duas, escrever teste que falhe: agent+writ de terceiro = 403. ► trecho de código

C3-02 Média

```
src/server/routeFactory.ts:84-89
migrations/0018_add_assigned_user_to_contact_deal.sql:12
migrations/0018_add_assigned_user_to_contact_deal.sql:15
migrations/0003_crm.sql:66
migrations/0003_crm.sql:99
```

checkOwnership falha aberto quando o registro não tem dono — e as migrations criam exatamente esse caso

Mesmo nas quatro tabelas com dono, a checagem cede quando o campo está vazio: ``if (!ownerId) return null``. O comentário assume que "sem dono definido" é um estado legítimo, mas o schema garante o oposto — ``DEFAULT`` em `contacts`, `deals`, `tasks` e `conversations` significa que todo registro criado sem atribuição explícita nasce sem dono. Um registro desses é gravação livre para qualquer ``agent``: a proteção existe no código e não dispara no dado. O efeito é inverso ao esperado: quanto mais antigo o registro (criado antes de a atribuição ser obrigatória, ou por um caminho que não a preenche), menos protegido ele está. A condição é fácil de alcançar — basta um contato importado.

Explorabilidade: Registro com ``assignedUserId`/`assigneeUserId`` vazio + PUT por ``agent`` ≠ dono → 200. O silêncio da checagem não é logado.

Correção: Falhar fechado é a opção simples e alinhada ao resto do app: ``if (!ownerId) retornar 403 nao_autorizado`` para papéis abaixo de manager. Se a intenção for tratar "sem dono" como posse da empresa (legítimo em CRM com designação opcional), então o caminho certo é tornar a atribuição obrigatória no schema (``NOT NULL`` sem default, com backfill) — e não deixar o default vazio decidir a segurança. Registrar a decisão como invariante, com teste que falhe para linha sem dono.

► [trecho de código](#)

C3-03 Média

src/server/routeFactory.ts:176-183
src/server/crud.ts:57-65
src/server/tables.ts:169-175

PUT aceita campos de autoria e atribuição no corpo: agente se autoatribui e falsifica autoria

O patch do PUT é repassado ao merge sem allowlist própria — a única restrição é a coluna existir no `shape`. Como os shapes incluem justamente os campos de autoria e atribuição, três jogadas ficam disponíveis para um `agent` numa tabela sem dono: (a) `assignedUserId` no próprio corpo, autoatribuindo o registro; (b) `createdBy`/`authorUserId` em qualquer tabela, falsificando autoria; (c) em `messages`, `text` + `fromMe` + `direction`, reescrevendo a mensagem como se fosse própria. O ponto importante é que (a) e (b) continuam disponíveis mesmo nas tabelas COM dono (C3-01 fecha só a cobertura): o gate de posse compara o dono persistido com o usuário, mas nada impede o mesmo PUT de trocar o dono junto. Um agente pode tomar um contato de outro, em uma requisição só, sem nenhum 403.

Explorabilidade: Sessão `agent` + PUT com `assignedUserId`/`assigneeUserId`/`createdBy` no corpo. Em tabela sem dono: 200 sempre. Em tabela com dono: 200 se a posse atual permitir (inclusive linha sem dono, por C3-02).

Correção: Derivar `assignedUserId`/`assigneeUserId` do `locals.user.id` no servidor (ignorar o valor do corpo), e remover `createdBy`/`authorUserId`/`fromMe` da superfície de escrita — `createdBy` deve ser preenchido na criação, nunca editado; `fromMe` só muda por caminho de sistema (ingest de webhook). Para o PUT, a forma mais barata de garantir isso é uma lista de campos imutáveis por tabela, checada antes do merge, com teste que falhe se um PUT conseguir mudar `createdBy`.

[▶ trecho de código](#)

4. Chaves expostas (hardcode)

ID	SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
C4-01	Crítica	docs/security-audit/dados_auditoria.py:433, 1354, 1358 (blob e5cd231) .dev.vars:9 tests/spec-v2/c-settings-secrets.test.ts:232-259	Segredo real da WAHA dentro de arquivo de texto versionado (docs/security-audit/dados_auditoria.py) O guardião do próprio repositório reagiu: `npm test` falha em `c-settings-secrets.test.ts` (AC-335), apontando `docs/security-audit/dados_auditoria.py` como arquivo rastreado que carrega a chave real. O relatório de auditoria vazava a credencial que ele deveria auditar — o arquivo de achados citava o segredo em claro para provar a existência dele. Isso é pior que tê-lo só em `.dev.vars`: `.dev.vars` é ignorado por design (AC-334 passa), enquanto um arquivo de documentação está no histórico do git. A chave aparece em: esta árvore de trabalho, todos os clones, qualquer backup do repositório, e o histórico — onde não basta apagar o arquivo, porque o conteúdo anterior permanece recuperável. Explorabilidade: Qualquer pessoa ou processo com leitura do repositório (ou de um clone dele) obtém a credencial da API WAHA. Não é explorável por requisição HTTP — é exposição por leitura de repositório. Correção: Rotacionar a WAHA_API_KEY primeiro — é o que tira o valor do histórico; só depois purga. Depois: (a) reescrever este arquivo citando apenas `arquivo:linha` com valor redigido (feito nesta rodada); (b) nunca colar segredo em texto versionado, nem para provar um achado — `git grep` do prefixo no histórico basta como evidência; (c) reescrever o histórico (`git filter-repo`) ou, no mínimo, registrar que a chave antiga está morta. O teste AC-335 já é a trava: ele falhou exatamente como deveria. ▶ trecho de código

C4-02	Alta	<code>.dev.vars:9</code> <code>.gitignore:5</code>	Credencial real da WAHA presente na árvore de trabalho (.dev.vars) <code>.dev.vars`</code> guarda a <code>`WAHA_API_KEY`</code> real em texto plano na máquina de desenvolvimento. O arquivo está corretamente ignorado e não rastreado (AC-334 passa), então não é um vazamento de repositório — mas é um segredo reproduzível em qualquer backup da máquina, snapshot de disco, ou sessão de shell que leia o arquivo. O relatório antigo tratava este item como achado de exposição; hoje o enquadramento correto é: material de segredo presente, contenção por filesystem em vez de por gestão de segredo. A severidade só cai depois da rotação de C4-01, porque a mesma chave também está no histórico do git. Explorabilidade: Leitura do arquivo por processo/usuário na mesma máquina. Sem exposição de rede: o arquivo não é servido. Correção: Rotacionar junto com C4-01 e mover a credencial para um cofre ou <code>`wrangler secret put`</code>, mantendo em <code>.dev.vars`</code> apenas o ponteiro não-sensível. Se o arquivo precisa existir localmente, documentar a CONTENTS_DENY por diretório (600) e a proibição de anexá-lo a ticket/bug report. ► trecho de código
C4-03	Média	<code>waha/.env:12</code> <code>.gitignore:7</code>	Histórico do git carrega hash de credencial WAHA (waha/.env em commits antigos) Nove commits tocaram <code>`waha/.env`</code>, entre eles <code>`882b050`</code>, <code>`59d917b`</code> ("T-103 feature: segredo da WAHA fora do controle de versao"), <code>`563bbf2`</code>, <code>`34829fe`</code>, <code>`46f03b2`</code>, <code>`588a256`</code>, <code>`0ce4f2d`</code>, <code>`8aab53d`</code>, <code>`484c891`</code> e <code>`90dfd83`</code>. O conteúdo versionado é o hash SHA-512 da chave — não a chave em si, mas um derivado que permite verificação offline de candidatos e confirma qual credencial era a válida na época. O arquivo não está mais rastreado e <code>.gitignore`</code> cobre <code>.env`</code>, então a exposição atual é só histórica; mas histórico é o armazenamento que ninguém limpa sozinho. Explorabilidade: Qualquer pessoa com um clone do repositório (inclusive shallow=false) lê o hash; brute-force de chave candidata é verificável offline sem rede. Correção: Rotacionar a credencial (o hash só é inofensivo depois que o segredo que ele resume está morto) e reescrever o histórico removendo <code>`waha/.env`</code> de todos os commits (<code>`git filter-repo --path waha/.env --invert-paths`</code>), com force-push coordenado. Se reescrever histórico for inviável, registrar explicitamente que o hash está comprometido e que a rotação é obrigatória — sem isso, o achado se repete a cada clone. ► trecho de código
C4-04	Média	<code>.dev.vars.example:33-35</code> <code>src/server/wahaWebhook.ts:51-57</code>	Webhook WAHA nasce com verificação de assinatura desligada: o template entrega segredo e flag vazios A leitura de configuração agora é correta — placeholder e segredo curto são recusados —, mas o valor default do arquivo de exemplo continua sendo "sem segredo e sem exigência". Quem copia <code>.dev.vars.example`</code> para <code>.dev.vars`</code> e não preenche esses dois campos (o caminho natural, já que o resto do arquivo vem preenchido) sobe o receiver com a assinatura desligada, e o comentário do próprio template diz isso. O ponto é o default: a proteção depende de o operador fazer a parte difícil, e o estado inicial é o inseguro. Explorabilidade: Configuração não editada → qualquer POST em <code>/api/whatsapp/webhook`</code> é aceito e ingerido no CRM, sem autenticação.</p> <p><b>Correção:</b> Trocar o default por seguro: <code>`WAHA_WEBHOOK_REQUIRE_SIGNATURE="true"`</code> no template, com <code>`WAHA_HMAC_SECRET`</code> claramente marcado como obrigatório, e fazer o receiver recusar o startup (não o request) quando o modo estrito não tiver segredo — hoje a recusa acontece por request, então a falha é silenciosa e o sintoma é "a verificação simplesmente não existe". Se o modo aberto for necessário em desenvolvimento, que seja explícito (<code>`WAHA_WEBHOOK_REQUIRE_SIGNATURE="false"`</code> comentado como só-dev). ► trecho de código

C4-05	Média	src/server/wahaWebhook.ts:33-34 src/server/wahaWebhook.ts:69-86 src/server/wahaWebhook.ts:44-49	fail-open na exigência de assinatura: modo estrito sem segredo utilizável aceita payload sem header O comentário das linhas 33-34 afirma fail-closed, e o código faz o oposto no caminho que importa. O ponto de recusa sem assinatura exige as DUAS condições: <code>`config.requireSignature && config.hmacSecret`</code>. Se o operador liga o modo estrito mas o segredo é recusado por <code>`usableSecret`</code> (placeholder, curto, ou vazio), <code>`hmacSecret`</code> é <code>`null`</code>, a conjunção falha e a execução cai no <code>`return { ok: true }`</code> final. Ou seja: a configuração que o operador entende como "mais segura" resulta em nenhuma verificação. O caminho de assinatura presente está correto (<code>`:75`</code> nega sem segredo, <code>`:77`</code> compara em tempo constante) — o defeito é exclusivamente a combinação <code>require+secret</code>, que transforma erro de configuração em autenticação desligada em vez de erro visível. Agrava C4-04: o template já entrega o modo estrito desligado, então o operador que liga a flag e esquece o segredo cai silenciosamente no modo aberto. Explorabilidade: <code>`WAHA_WEBHOOK_REQUIRE_SIGNATURE=true`</code> + segredo ausente/placeholder/curto + POST sem header <code>`x-webhook-hmac`</code> → aceito, arquivado e ingerido no CRM. Correção: Inverter a lógica para que a recusa dependa só da flag: <code>`if (config.requireSignature && !signature) return { ok: false, reason: 'missing_signature' }`</code> — assim modo estrito sem segredo utilizável recusa todo request (fail-closed), que é o comportamento prometido pelo comentário. Complementar com erro explícito no startup e com a troca do teste existente em <code>`tests/server/wahaWebhook.test.ts:102`</code> (hoje afirma que <code>`requireSignature: true`</code> com <code>`hmacSecret: null`</code> aceita) — esse teste é a prova viva do fail-open e precisa mudar junto. ► trecho de código
C4-06	Baixa	.github/workflows/ci.yml:142-144	npm audit não bloqueia a CI (continue-on-error: true) O job de auditoria roda e o resultado é ignorado: <code>`continue-on-error: true`</code> faz o GitHub Actions marcar o step como sucesso mesmo com vulnerabilidade high/critical, então a verificação não tem efeito de gate. Não é uma vulnerabilidade por si só — é um controle que não controla, o que é pior que não tê-lo, porque o relatório de CI mostra o job como verde. Explorabilidade: Qualquer vulnerabilidade high/critical em dependência: a CI segue verde. Correção: Remover <code>`continue-on-error`</code> para <code>`npm audit --audit-level=high`</code>, ou trocar por <code>`continue-on-error: true`</code> com upload de artifact do relatório e uma condição de falha separada no nível critical. Como dependência é superfície de alto retorno para esta categoria, o passo só é útil se mandar em gate. ► trecho de código
C4-07	Baixa	src/server/seedCredential.ts:36-43 src/worker.ts:12-25	Nenhuma validação de configuração no startup: assertNoSeedCredential não é chamada pelo Worker Existe uma função que falha fechado enquanto há credencial de seed reproduzível, com mensagem de erro explícita — e ela só é chamada de dentro do próprio script de bootstrap e dos testes (<code>`grep`</code> de consumidores: <code>`bootstrap-admin.ts:59,87`</code> e <code>`tests/spec-v2/c-settings-secrets.test.ts:135-141`</code>; nenhuma em <code>`src/server`</code> nem em <code>`worker.ts`</code>). O endpoint do Worker sobe sem verificar nada. O mesmo vale para a configuração do webhook: um segredo ausente ou inválido não produz erro de boot, apenas degradação silenciosa em runtime (C4-04, C4-05). A fechamento do achado de senha de seed do relatório antigo é real — o seed não grava mais hash reproduzível, <code>`bootstrap-admin.ts:26-32`</code> sorteia a senha com <code>`crypto.getRandomValues`</code>, e o middleware bloqueia a API enquanto <code>`mustChangePassword=1`</code> — mas o fail-closed que protects o estado legado não está ligado ao processo que decide subir. Explorabilidade: Deploy com credencial de seed legada ou com webhook em modo aberto: o Worker sobe normalmente e o problema só aparece em um request. Correção: Chamar <code>`assertNoSeedCredential(env.DB)`</code> no boot do Worker (uma vez, com cache por isolate ou marcado no bundle) e falhar o deploy quando ela lançar. Adicionar, na mesma função, a validação de configuração do WAHA (chave presente, <code>`WAHA_HMAC_SECRET`</code> válido se <code>`REQUIRE_SIGNATURE=true`</code>). Alternativa mais leve: um <code>`npm run preflight`</code> obrigatório no pipeline, com checagem que rode contra o D1 remoto — desde que o pipeline não seja opcional. ► trecho de código

5. Inputs sem tratamento (XSS)

ID	SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
C5-01	Baixa	src/domain/format.ts:33-37 src/domain/format.ts:39-43 src/ui/views/crm/crmUi.ts:57, 69, 77, 83, 93-94, 131	escapeHtml não escapa aspas: helper disponível para o contexto errado (latente, sem sink hoje) `escapeHtml` escapa `&`, `<` e `>` via `textContent` → `innerHTML`, mas não aspas — corretamente, porque é um helper de contexto de texto. O risco é latente: `escapeAtrib` existe e cobre o caso de atributo (`crmUi.ts:57,69,77,83,131`), e hoje nenhum sink de atributo usa o helper base, portanto não há injeção viva. O que fica registrado é a superfície: qualquer novo `value="\${escapeHtml(x)}"` reproduz a quebra de atributo, e o nome do helper não avisa. Agrava o escopo: com sessão em cookie HttpOnly (C1-02), mesmo um eventual breakout não vira roubo de token. Os sinks reais da rodada anterior estão corrigidos: `Modal.ts:32` escapa `options.title`, `DashboardView.ts:88` escapa `i.name`/`i.unit`, `LoginView.ts:46-47` escapa `me.name`/`me.email`, e a CSP do middleware (`script-src 'self'`, sem `unsafe-inline`) contém o resto. Explorabilidade: Não explorável no código atual: nenhum atributo é preenchido com escapeHtml. Exige uma futura interpolação em atributo. Correção: Fechar a classe de falha, não só o helper: renomear `escapeHtml` para `escapeText` (nome que declara o contexto) e fazer `escapeAtrib` ser o único aceito em atributo, com um teste que varra os arquivos de `src/ui` e falhe se encontrar `="\${escapeText(` ou `escapeHtml` dentro de atributo. Enquanto isso, documentar no helper que ele NÃO é seguro em atributo. ► trecho de código
C5-02	Informativa	src/middleware.ts:31-56 src/server/mapping.ts:52-58 src/server/crud.ts:46, 65, 78 src/domain/format.ts:33-43	Verificação positiva: CSP restritiva, sinks escapados e anti-SQLi por allowlist de coluna A camada de escape está coerente onde importa, e a defesa não depende só disso: a CSP sem `unsafe-inline` impede execução de script injetado mesmo que um breakout de atributo ocorra, e o allowlist de coluna com `bind()` mantém SQL fora do alcance do corpo JSON (nome de coluna não pode ser contrabandeado como chave). Combinadas com o cookie HttpOnly (C1-02), as três camadas independentes reduzem um erro de escaping futuro a um incidente de conteúdo, não a comprometimento de sessão. Explorabilidade: Não explorável: controle verificado, não ausência de controle. Correção: Nenhuma. Preservar a separação dos helpers de escape por contexto. ► trecho de código

Recomendações priorizadas

P1 fecha o vetor de comprometimento total (escalada de privilégio e vazamento de dados). **P2** fecha IDOR e injeção de HTML. **P3** é higiene e defesa em profundidade — os itens que reduzem a superfície e impedem a reintrodução das mesmas classes de falha.

PRIO.	ACHADOS	AÇÃO	POR QUÊ
P1	C4-01 C4-02 C4-03	Rotacionar a WAHA_API_KEY (uma vez, cobre as três), reescrever este arquivo sem segredo em claro — feito nesta rodada — e purgar `waha/.env` do histórico com git filter-repo.	Segredo em texto versionado e em histórico é o único achado desta rodada que compromete fora do perímetro da aplicação: qualquer clone tem a credencial. Rotacionar invalida o que já vazou; reescrever o arquivo impede a repetição.
P1	C3-01	Subir o piso de `update` para `manager` nas tabelas sensíveis sem dono (messages, orders, calendar-events, conversation-notes, catalog-products) e modelar `assignedUserId` nas demais tabelas de trabalho do CRM, registrando em `OWNER_FIELDS`.	15 das 19 rotas de item não têm gate de posse nenhum; o piso `agent` de C2-02 transformou isso em IDOR(truthy) para o papel mais comum do time.
P1	C5-02	Preservar a separação escapeText/escapeAtrib e a CSP sem `unsafe-inline`.	São as duas camadas que mantêm um erro de escaping futuro como incidente de conteúdo, não como comprometimento de sessão.
P2	C3-02	Falhar fechado em `checkOwnership` quando o registro não tem dono (403 para papel abaixo de manager), ou tornar a atribuição obrigatória no schema com backfill.	`DEFAULT ""` nas migrations garante linhas sem dono, e o código trata "sem dono" como permissão — quanto mais antigo o registro, menos protegido.
P2	C3-03	Derivar `assignedUserId`/`assigneeUserId` do `locals.user.id` no servidor e remover `createdBy`/`authorUserId`/`fromMe` da superfície de escrita, com lista de campos imutáveis por tabela.	Autoatribuição e falsificação de autoria continuam disponíveis mesmo nas tabelas com dono, porque o PUT aceita trocar o dono junto com o resto do registro.
P2	C4-04 C4-05	Trocar o default do template para `REQUIRE_SIGNATURE="true"` e inverter a lógica de C4-05 para que o modo estrito recuse quando o segredo não é utilizável — atualizando o teste que hoje afirma o comportamento fail-open.	Um operador que liga a flag e esquece o segredo acaba com a verificação inexistente, e o comentário do código promete o contrário do que ele faz.
P2	C5-01	Renomear `escapeHtml` para `escapeText` e adicionar teste que falhe se o helper de texto aparecer dentro de um atributo em `src/ui`.	Fecha a classe de falha por contexto, não só o helper: o nome atual não avisa o próximo autor, e nenhum sink vivo existe hoje para servir de trava.
P3	C4-07	Chamar `assertNoSeedCredential(env.DB)` e a validação de config do WAHA no boot do Worker (ou em um preflight obrigatório do pipeline).	O fail-closed existe e é correto; só não está ligado ao processo que sobe a aplicação, então erro de configuração vira degradação silenciosa em runtime.
P3	C4-06	Remover `continue-on-error` do `npm audit` ou condicionar a falha em nível critical.	Um controle que não controla é pior que a ausência dele: o relatório de CI mostra verde onde há vulnerabilidade.
P3	C1-01	Registrar como invariante (teste) que o recorte LGPD nunca devolve linha sem atribuição, e que rota nova não pode assumir `listEntities` restrita.	A listagem global é decisão de projeto legítima; o risco é ela ser tratada como garantia por quem escreve a próxima rota.

Issues para o GitHub

Texto completo de cada issue, pronto para copiar e colar. Cada bloco delimitado por `--- ISSUE n ---` e `--- FIM ISSUE n ---`. Achados triviais relacionados foram agrupados em uma issue única (issues 7, 9 e 12) para não gerar spam.

ISSUE 1
<pre>--- ISSUE 1 --- # [Segurança] Segredo real da WAHA em arquivo versionado e no histórico do git **Labels sugeridas:** `security` · `critical` · `secrets` · `git-history` ## Problema A credencial `WAHA_API_KEY` real está em três lugares. **(a) Em arquivo rastreado.** `docs/security-audit/dados_auditoria.py` – o arquivo de dados do próprio relatório de auditoria – continha o valor em claro, para provar o achado que descrevia. É texto versionado: está no histórico do git, em todo clone, em todo backup do repositório. **(b) Na árvore de trabalho.** `.dev.vars:9` guarda o mesmo valor. O arquivo está corretamente ignorado (AC-334 passa), então aqui a exposição é por leitura de disco, não por repositório. **(c) No histórico.** `waha/.env:12` foi versionado em nove commits e contém o hash SHA-512 da credencial – não a chave, mas um derivado que permite verificação offline de candidatos. O achado é que a auditoria de segurança estava vazando o segredo que ela audita: o teste de regressão do próprio repositório (`c-settings-secrets.test.ts`, AC-335) reprovou por causa disso, e está reprovando até que o arquivo seja corrigido. ## Evidência ```ts tests/spec-v2/c-settings-secrets.test.ts (AC-335, em falha) expect([file, leaked]).toEqual([file, []]); // file = docs/security-audit/dados_auditoria.py waha/.env:12 (em commits antigos; hoje não rastreado) WAHA_API_KEY_SHA512=sha512:<redigido> .dev.vars:9 (untracked, .gitignore:5) WAHA_API_KEY="<redigido>" .gitignore:5,7 .dev.vars .env commits que tocaram waha/.env: 882b050 59d917b 563bbf2 34829fe 46f03b2 588a256 0ce4f2d 8aab53d 484c891 90dfd83 ``` ## Impacto Comprometimento da credencial da API WAHA, e portanto do canal de WhatsApp da operação: quem lê a chave pode chamar a API do engine (listar e manipular sessões, enviar mensagens em nome da empresa). Não é uma falha explorável por requisição anônima – é exposição por leitura de repositório, o que implica qualquer pessoa com acesso ao código, CI, dependência com persistência de artefato, ou cópia do projeto. Rotacionar é obrigatório mesmo após remover o arquivo do working tree, porque o histórico preserva o valor. ## Sugestão de correção 1. Rotacionar a WAHA_API_KEY no engine (invalida tudo que já vazou, histórico incluído). 2. Reescrever qualquer arquivo versionado que cite o segredo, substituindo por `arquivo:linha` com valor redigido – este relatório já vem corrigido. 3. Purgar `waha/.env` do histórico: `git filter-repo --path waha/.env --invert-paths`, com force-push coordenado. Se reescrever for inviável, registrar que o hash está comprometido e que a rotação é obrigatória. 4. Nunca colar segredo em texto versionado, nem como evidência: `git log -S` do prefixo basta para provar o achado. ## Critérios de aceite - [] A WAHA_API_KEY atual é diferente da que aparece em qualquer commit (verificável com git log -S do prefixo antigo). - [] npm test passa, incluindo AC-335, sem que nenhum arquivo rastreado carregue a chave. - [] Nenhum arquivo rastreado contém valor de segredo; citações são arquivo:linha com valor redigido. - [] `git log --all -- waha/.env` não retorna commits (histórico purgado) ou há registro explícito de que o hash foi commitment-rotacionado. - [] .dev.vars continua não rastreado (AC-334) e a credencial local vem de cofre/wrangler secret, não de arquivo de texto. --- FIM ISSUE 1 ---</pre>
ISSUE 2
<pre>--- ISSUE 2 --- # [Segurança] IDOR: agent+ altera registro de terceiro em 15 das 19 rotas de item **Labels sugeridas:** `security` · `high` · `idor` · `broken-access-control` · `authorization` ## Problema `routeFactory.ts` tem um gate de posse (`checkOwnership`), mas ele só se aplica às tabelas listadas em `OWNER_FIELDS` – hoje quatro: `contacts`, `conversations`, `deals`, `tasks`. Das dezenove rotas de item expostas por `createItemRoutes`, as outras quinze passam por `if</pre>

```
(!ownerField) return null; // Sem campo de dono, permite`.
```

Desde que o piso de escrita passou a ser `agent` (o papel que o time de vendas realmente usa), a consequência é direta: qualquer `agent` faz PUT em qualquer registro de `activities`, `appointment-types`, `calendar-events`, `catalog-products`, `conversation-notes`, `messages`, `pipelines`, `quick-replies`, `stages`, `tags`, e nas tabelas do ERP (`components`, `customers`, `ingredients`, `orders`, `products`), bastando conhecer o id.

O caso mais grave é `messages`. `MESSAGES\_SHAPE` inclui `text`, `fromMe`, `direction`, `createdBy`, `conversationId`, `externalId`, `remoteJid`. Não é apenas leitura indevida de registro alheio: é **reescrita de mensagem já entregue**, com marcação de direção e autoria forjadas.

Evidência

```
```ts
src/server/routeFactory.ts:62-69
const OWNER_FIELDS: Record<string, string> = {
 contacts: 'assignedUserId',
 conversations: 'assignedUserId',
 deals: 'assignedUserId',
 tasks: 'assigneeUserId',
};

src/server/routeFactory.ts:81-82
const ownerField = OWNER_FIELDS[table];
if (!ownerField) return null; // Sem campo de dono, permite

src/server/routeFactory.ts:29-34 (piso que torna isso alcançável)
const DEFAULT_ROLES: Required<RoleConfig> = {
 list: 'viewer', create: 'agent', update: 'agent', delete: 'manager',
};

src/pages/api/crm/messages/[id].ts:1-3 (sem RoleConfig, sem dono)
import { createItemRoutes } from '../.../server/routeFactory';
import { MESSAGES_TABLE, MESSAGES_SHAPE } from '../.../server/tables';
export const { PUT, DELETE } = createItemRoutes(MESSAGES_TABLE, MESSAGES_SHAPE);

src/server/tables.ts:169-175 (superfície de escrita)
columns: [
 'ack', 'conversationId', 'createdAt', 'createdBy', 'deliveredAt', 'direction',
 'editedAt', 'externalId', 'fromMe', 'id', 'mediaMime', 'mediaUrl',
 'messageType', 'readAt', 'remoteJid', 'revokedAt', 'text', 'waStatus', ...
],
...
boolFields: ['fromMe'],
...
```
```

Impacto

Um `agent` – o papel mais comum – pode alterar dados de qualquer colega e, em `messages`, reescrever o histórico de conversa do inbox e forjar autoria/direção. Como `DELETE` exige `manager`, o dano é de integridade, não de disponibilidade: o registro some de vista, mas a fraude fica. Para o inbox, whose messages são a evidência do relacionamento com o cliente, isso éb worse do que ler o que não deveria.

Sugestão de correção

1. Curto prazo, sem migration: declarar `RoleConfig` por tabela nas rotas sensíveis, com `update: 'manager'` para `messages`, `orders`, `calendar-events`, `conversation-notes` e `catalog-products`.
2. Longo prazo: adicionar `assignedUserId` às tabelas de trabalho do CRM (`activities`, `conversation-notes`, `catalog-products`, `stages`, `tags`) com migration e entrada em `OWNER\_FIELDS`; para `messages`, resolver posse pela conversa (`conversationId` → `conversations.assignedUserId`) em vez de coluna própria.
3. Escrever o teste que hoje falha: agent + PUT em registro de terceiro = 403, para todas as tabelas, e não só para as quatro com dono.

Critérios de aceite

```
- [ ] PUT /api/crm/messages/[id] por sessão agent em mensagem de conversa não atribuída retorna 403.
- [ ] Não existe rota de item reachable por agent que aceite escrita sem gate de papel ou de posse (teste que varre
src/pages/api/**/[id].ts falha se alguma aparecer).
- [ ] messages: texto, direção, fromMe e createdBy não são alteráveis por PUT de agente em nenhuma tabela.
- [ ] Toda tabela com coluna de dono tem entrada em OWNER_FIELDS, ou não tem coluna de dono.
- [ ] Teste de tabela que falhe para qualquer tabela nova em createItemRoutes sem RoleConfig explícito quando o piso padrão exigir agente+em dado sensível.
--- FIM ISSUE 2 ---
```

ISSUE 3

```
--- ISSUE 3 ---
```

[Segurança] checkOwnership falha aberto quando o registro não tem dono, e o schema garante esse caso

****Labels sugeridas:**** `security` · `medium` · `authorization` · `idor` · `data-model`

Problema

Mesmo nas quatro tabelas com dono, a checagem cede quando o campo está vazio:

```
```ts
const ownerId = (entity as Record<string, unknown>)[ownerField];
if (!ownerId) return null; // Sem dono definido, permite
```
```

O comentário assume que "sem dono definido" é estado legítimo. O schema garante o contrário: `DEFAULT` em `contacts` e `deals` (migration 0018), `tasks.assigneeUserId` (0003) e `conversations.assignedUserId` (0003). Todo registro criado sem atribuição explícita nasce sem dono – e nesse estado a proteção existe no código e não dispara no dado.

O efeito é invertido ao que se espera de um controle de posse: quanto mais antigo o registro (ou criado por um caminho que não preenche atribuição), menos protegido. A condição é trivial de alcançar – um contato importado, uma conversa criada pelo webhook sem atribuição.

Evidência

```
```ts
src/server/routeFactory.ts:84-89
 const entity = await getEntity(db, table, shape, id);
 if (!entity) return notFound();

 const ownerId = (entity as Record<string, unknown>)[ownerField] as
 string | undefined;
 if (!ownerId) return null; // Sem dono definido, permite

migrations/0018_add_assigned_user_to_contact_deal.sql:12,15
ALTER TABLE contacts ADD COLUMN assignedUserId TEXT NOT NULL DEFAULT '';
ALTER TABLE deals ADD COLUMN assignedUserId TEXT NOT NULL DEFAULT '';

migrations/0003_crm.sql:66 (tasks)
 assigneeUserId TEXT NOT NULL DEFAULT '',

migrations/0003_crm.sql:99 (conversations)
 assignedUserId TEXT NOT NULL DEFAULT '',
```
```

Impacto

Escritura por `agent` em qualquer registro sem dono, de forma silenciosa: o código de autorização decide que pode, o resultado é 200, e não há log registrando que a checagem foi omitida. Como a linha não tem dono, também não existe rastro que permita dizer depois quem poderia ter escrito.

Sugestão de correção

Escolher explicitamente uma das duas semânticas e implementá-la no schema, não no default.

(a) Falhar fechado: `if (!ownerId) return json({ error: 'nao\_authorized' }, 403)` para papéis abaixo de manager. Correção de uma linha, sem migration, e é o que o resto do app já faz para segredo recusado.

(b) Atribuição obrigatória: `NOT NULL` sem default, com backfill preenchendo `assignedUserId` a partir do primeiro admin, e atribuição obrigatória na criação. Mais fiel ao modelo, mais caro de migrar.

Independente da escolha: log quando a checagem de posse for omitida por falta de dono, para que o dado sem dono apareça.

Critérios de aceite

- [] Registro com assignedUserId/assigneeUserId vazio não pode ser alterado por agent (403), ou a semântica escolhida está documentada e testada.
 - [] Existe teste que cria explicitamente uma linha sem dono e demonstra o comportamento escolhido.
 - [] Nenhuma migration nova cria coluna de dono com DEFAULT ''.
 - [] A omissão da checagem por falta de dono gera entrada de log/auditoria.
- FIM ISSUE 3 ---

ISSUE 4

--- ISSUE 4 ---

[Segurança] PUT aceita campos de atribuição e autoria: autoatribuição e falsificação de autoria

Labels sugeridas: `security` · `medium` · `mass-assignment` · `idor` · `authorization`

Problema

O `updateHandler` repassa o corpo do PUT direto ao merge, e a única restrição é a coluna existir no `shape`. Como os shapes incluem os campos de atribuição e autoria, três jogadas ficam disponíveis:

(a) Autoatribuição. `PUT` com `assignedUserId: <meu id>` no corpo move o registro para si – inclusive nas quatro tabelas que *têm* dono, porque o gate compara o dono persistido, e o dono novo vai na mesma requisição. Um agente toma o contato de outro em uma chamada, sem nenhum 403.

(b) Falsificação de autoria. `createdBy`/`authorUserId` são colunas comuns nos shapes e são graváveis: um agente escreve que a mensagem ou a atividade foi criada por outra pessoa.

(c) Reescrita de direção. Em `messages`, `text` + `fromMe` + `direction` mudar juntos transforma mensagem recebida em enviada, ou o contrário – compartilhado com a issue 2, mas com vetor próprio (mass assignment, não falta de posse).

Evidência

```
```ts
src/server/routeFactory.ts:176-183
 const patch = (await context.request.json()) as {
 id?: string;
 } & Record<string, unknown>;
 const invalid = refuseInvalid(shape, patch);
 if (invalid) return invalid;
 const saved = await updateEntity(
 getDb(), table, shape, context.params.id!, patch
);

src/server/crud.ts:59-65 (merge sem allowlist própria)
 const merged = { ...existing, ...patch } as T;
```

```
const row = entityToRow(merged as Record<string, unknown>, shape);
const { sql, values } = buildUpdate(table, id, row, shape.columns);

src/server/tables.ts:169-175 (createdBy, fromMe, direction são colunas do shape)
columns: ['ack', 'conversationId', 'createdAt', 'createdBy', 'deliveredAt',
 'direction', ..., 'fromMe', ..., 'text', ...],
boolFields: ['fromMe'],
...
```

**## Impacto**

Autoatribuição contorna a única proteção de posse que existe (issue 3) sem precisar de nenhuma condição especial – basta o gate de posse ver o dono antigo. Falsificação de autoria quebra a trilha de auditoria do CRM: `action\_log` registra quem fez a operação, mas o registro aponta outra pessoa como autora, e as duas informações passam a discordar sem sinal.

**## Sugestão de correção**

Separar campo de criação de campo de manutenção:

- `assignedUserId`/`assigneeUserId`: ignorar o valor do corpo e derivar de `locals.user.id` (com um caso explícito de reatribuição por manager/admin, que é operação legítima de gestão).
- `createdBy`/`authorUserId`: preenchidos na criação, nunca aceitos no PUT. - `fromMe`/`direction`: mudados apenas por caminho de sistema (ingest do webhook), não pela API de CRUD.

A forma mecânica de garantir é uma lista `IMMUTABLE\_FIELDS` por tabela verificada antes do merge, com teste que falhe para PUT tentando mudar `createdBy` em qualquer tabela.

**## Critérios de aceite**

- [ ] PUT com assignedUserId no corpo não altera o dono do registro (o valor vem do locals.user ou é ignorado).
- [ ] PUT com createdBy/authorUserId no corpo não altera a autoria registrada.
- [ ] PUT com fromMe/direction em messages retorna 400 ou é ignorado, fora do caminho de ingest.
- [ ] Existe lista de campos imutáveis por tabela e teste que cobre, no mínimo, createdBy e assignedUserId.

--- FIM ISSUE 4 ---

## ISSUE 5

--- ISSUE 5 ---

# [Segurança] Webhook WAHA: template nasce com assinatura desligada e o modo estrito falha aberto sem segredo utilizável

**\*\*Labels sugeridas:\*\*** `security` · `medium` · `webhook` · `fail-open` · `config-default`

**## Problema**

Duas falhas encostadas, com o mesmo efeito prático: nenhum request é autenticado por assinatura.

**\*\* (a) Default inseguro no template. \*\*** `dev.vars.example:34-35` traz `WAHA\_HMAC\_SECRET=""` e `WAHA\_WEBHOOK\_REQUIRE\_SIGNATURE=""`. Quem copia o template – o caminho natural, já que o resto vem preenchido – e não edita esses dois campos sobe o receiver com a verificação desligada. O comentário do próprio template avisa que vazio desliga.

**\*\* (b) fail-open na combinação flag+segredo. \*\*** Em `authenticateWahaWebhook`, a recusa de payload sem assinatura exige as duas condições:

```
```ts
if (config.requireSignature && config.hmacSecret) {
  return { ok: false, reason: 'missing_signature', ... };
}
return { ok: true, reason: 'ok', signatureVerified: false };
```
```

Se o operador liga `REQUIRE\_SIGNATURE=true` mas o segredo é recusado por `usableSecret` (placeholder, curto, vazio), `hmacSecret` é `null`, a conjunção falha e a execução cai no `return { ok: true }` final. A configuração que o operador entende como "mais segura" resulta em nenhuma verificação – e o comentário nas linhas 33-34 do mesmo arquivo afirma o oposto: "um segredo recusado DEIXA A VERIFICACAO DESLIGADA (fail-closed, nunca fail-open)".

O caminho de assinatura presente está correto (nega sem segredo, compara em tempo constante); o defeito é só a combinação.

**## Evidência**

```
```ts
dev.vars.example:33-35
# Um segredo de uso unico por ambiente: `openssl rand -hex 32`.
# Vazio DESLIGA a verificacao de assinatura; um segredo curto (<32) tambem e
# recusado e o receiver falha fechado. Copie este arquivo e troque o valor.
WAHA_HMAC_SECRET=""
WAHA_WEBHOOK_REQUIRE_SIGNATURE=""

src/server/wahaWebhook.ts:33-34 (o que o código promete)
// curtas sao recusados, e um segredo recusado DEIXA A VERIFICACAO DESLIGADA
// (fail-closed, nunca fail-open).

src/server/wahaWebhook.ts:44-49 (recusa de segredo inutilizavel)
function usableSecret(value: string | null): string | null {
  if (!value) return null;
  if (PLACEHOLDER_SECRETS.has(value.trim().toLowerCase())) return null;
  if (value.length < HMAC_MIN_BYTES) return null;
  return value;
}

src/server/wahaWebhook.ts:82-85 (o que o código faz)
if (config.requireSignature && config.hmacSecret) {
```

```

    return { ok: false, reason: 'missing_signature', signatureVerified: false };
  }
  return { ok: true, reason: 'ok', signatureVerified: false };
}

tests/server/wahaWebhook.test.ts:102 (a prova viva do fail-open)
const auth = await authenticateWahaWebhook(request(BODY, 'anything'),
  { hmacSecret: null, requireSignature: true });
// o teste afirma que o payload é aceito neste caso
...

## Impacto

Qualquer POST em `/api/whatsapp/webhook` é aceito, arquivado e ingerido no CRM sem autenticação. O que o atacante controla vai para o
inbox: mensagens, ticks de entrega e updates de status, com `externalId` e `remoteJid` forjáveis. Isso permite forjar conversas e
adulterar o histórico de delivery que o painel mostra. A rota é `PUBLIC_PATHS`, então nenhum middleware a protege – a assinatura é a única
fronteira.

## Sugestão de correção

1. Inverter a condição para que a recusa dependa só da flag:
  `if (config.requireSignature && !signature) return { ok: false, reason: 'missing_signature' }`. Modo estrito sem segredo utilizável
passa a recusar todo request, que é o fail-closed prometido.
2. Trocar o default do template para `WAHA_WEBHOOK_REQUIRE_SIGNATURE="true"`, marcando o segredo como obrigatório; se o modo aberto for
necessário em dev, que seja explícito e comentado.
3. Falhar no startup (não no request) quando o modo estrito não tiver segredo – hoje a recusa é por request, então o sintoma é "a
verificação não existe" em vez de um erro de configuração.
4. Atualizar o teste de `tests/server/wahaWebhook.test.ts:102`, que hoje consagra o comportamento fail-open.

## Critérios de aceite

- [ ] authenticateWahaWebhook com requireSignature=true e hmacSecret=null recusa payload sem assinatura (missing_signature).
- [ ] O teste que hoje afirma o contrário foi alterado, e há teste para assinatura presente + segredo ausente (bad_signature).
- [ ] .dev.vars.example traz REQUIRE_SIGNATURE=true e o segredo marcado como obrigatório.
- [ ] UmReceiver com modo estrito e segredo inválido falha no boot com erro de configuração, não apenas em request.
- [ ] POST sem header x-webhook-hmac no receiver em modo estrito não altera nenhum registro no D1.
--- FIM ISSUE 5 ---

```

ISSUE 6

```

--- ISSUE 6 ---
# [Segurança] Controles que não controlam: npm audit sem gate e assertNoSeedCredential fora do boot

**Labels sugeridas:** `security` · `low` · `ci` · `fail-open` · `configuration`

## Problema

Dois controles existem, estão escritos corretamente, e não produzem efeito porque nenhum ponto do sistema os invoca no momento em que
decides.

**(a) `npm audit` não bloqueia.** O step da CI roda `npm audit --audit-level=high` com `continue-on-error: true`. Vulnerabilidade high ou
critical produz step marcado como sucesso: o job aparece verde no relatório da CI. Um controle que não controla é pior que a ausência
dele, porque cria confiança falsa.

**(b) `assertNoSeedCredential` não é chamada no boot.** A função falha fechado enquanto existe hash de seed reproduzível, com mensagem
explícita – e seus únicos consumidores são o próprio script de bootstrap e os testes. Nenhuma chamada em `src/server`, nenhuma em
`worker.ts`. O endpoint sobe sem verificar nada. O mesmo vale para a configuração do webhook: segredo ausente não gera erro de boot,
vira degradação silenciosa em runtime (issue 5).

O fechamento do achado de senha de seed da auditoria anterior é real – o seed não grava hash reproduzível e `bootstrap-admin.ts:26-32`
sorteia a senha com `crypto.getRandomValues`. O que falta é o fail-closed que protege o estado herdado estar ligado ao processo que sobe a
aplicação.

## Evidência

```ts
.github/workflows/ci.yml:142-144
- name: Run npm audit
 run: npm audit --audit-level=high
 continue-on-error: true

src/server/seedCredential.ts:36-43
// Fail-closed: refuses to continue while a replayable admin credential exists.
export async function assertNoSeedCredential(db: Database): Promise<void> {
 const users = await listEntities<{ email: string; passwordHash: string }>(
 db, USERS_TABLE, USERS_SHAPE
);
 const offender = users.find((user) => isLegacySeedHash(user.passwordHash));
 if (offender) throw new SeedCredentialError(offender.email);
}

src/worker.ts:12-25 (o boot não verifica nada)
export default {
 async fetch(request: Request, env: Env, ctx: ExecutionContext) {
 const state = new FetchState(request);
 const asset = await cf(state, env, ctx);
 if (asset) return asset;
 return finalize(state, await astro(state));
 },
 async scheduled(_event: ScheduledEvent, env: Env, _ctx: ExecutionContext) {
 await runRetention(env.DB, env);
 }
}

```

```
}
};

grep -rn assertNoSeedCredential src/ scripts/ tests/
scripts/bootstrap-admin.ts:59,87
tests/spec-v2/c-settings-secrets.test.ts:135-141
(nenhuma em src/server nem em worker.ts)
````  
  
## Impacto  
  
Indireto, e é por isso que fica em baixa. (a) Um CVE high em dependência transita sem bloquear merge, e o sinal verde na CI é o oposto do que deveria comunicar. (b) Um deploy com credencial legada de seed, ou com webhook em modo aberto, sobe "normalmente": o problema só aparece quando alguém usa — e a defesa que existe para impedi-lo não está no caminho.  
  
## Sugestão de correção  
  
1. Remover `continue-on-error` do step de `npm audit` (ou manter o step tolerante e adicionar condição de falha separada para `critical`, com o relatório enviado como artifact).  
2. Chamar `assertNoSeedCredential(env.DB)` no boot do Worker — uma vez, com cache por isolate ou verificado no bundle — e falhar o deploy quando ela lançar. Na mesma passagem, validar a configuração do WAHA: `WAHA_API_KEY` presente e `WAHA_HMAC_SECRET` utilizável quando `REQUIRE_SIGNATURE=true`.  
3. Alternativa mais leve e mais garantida: um `npm run preflight` obrigatório no pipeline, com a checagem rodando contra o D1 remoto — desde que o pipeline não seja opcional nem esteja commented out.  
  
## Critérios de aceite  
  
- [ ] npm audit --audit-level=high falha o job quando há vulnerabilidade high (ou critical, conforme a política escolhida).  
- [ ] assertNoSeedCredential é chamada no boot do Worker / em preflight obrigatório, e o deploy falha quando existe hash legado.  
- [ ] Config ausente ou inválida do WAHA produz erro de boot, não apenas resposta 200 silenciosa em runtime.  
- [ ] Existe teste que falha se assertNoSeedCredential perder o único consumidor de produção.  
--- FIM ISSUE 6 ---
```